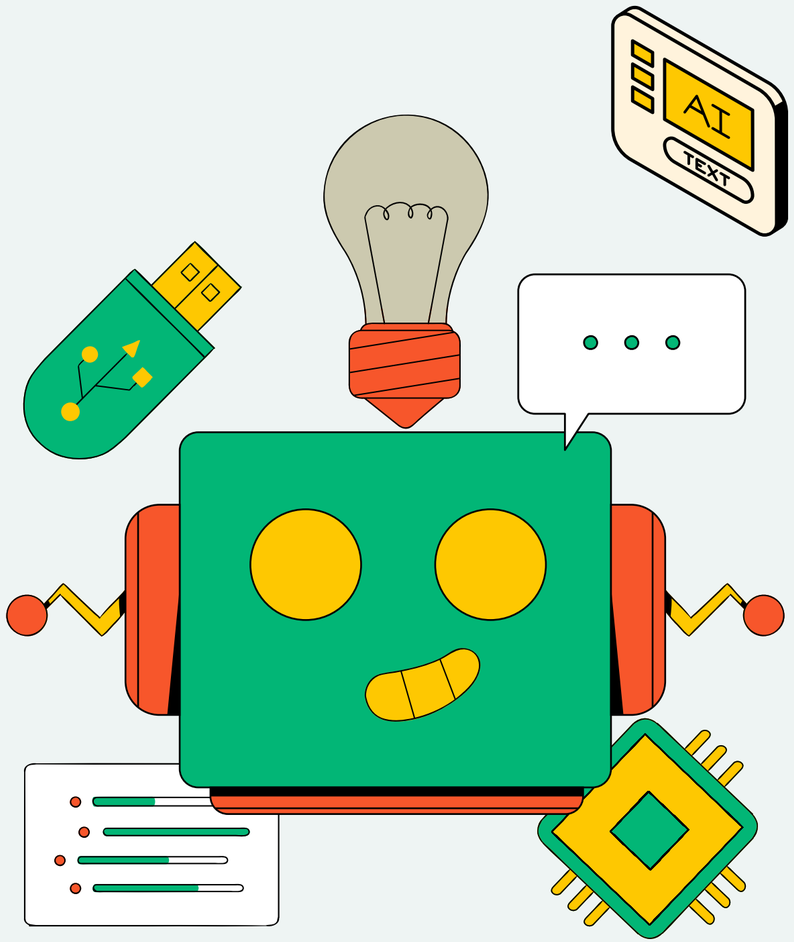


NEURAL RADIANCE FIELDS (NERF)

PRESENTATION

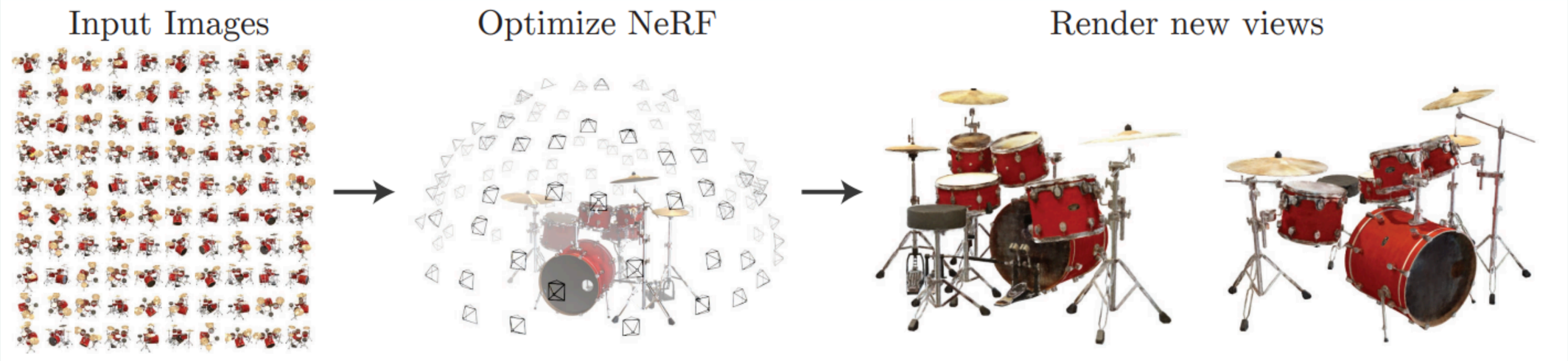
PRESENTE PAR :

BRYAN ANTHONY QUEEN

NINO SPORTES-MILOT

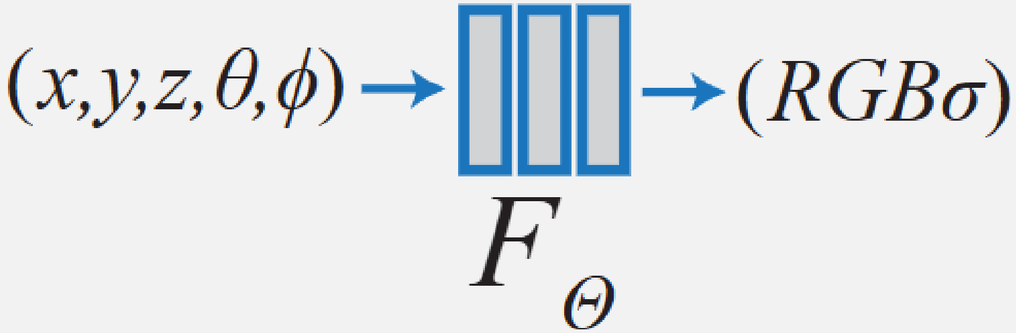
INTRODUCTION

Qu'est-ce que NeRF ?



NEURAL RADIANCE FIELD SCENE REPRESENTATION

- Objectif : représenter une scène 3D à partir d'un ensemble de photos 2D avec les poses de caméra



- Une scène = une fonction continue 5D : position **(x, y, z)** + direction de vue **(θ, φ)**

→ **(θ, φ)** exprimé par vecteur 3D **d**

$$F_\Theta : (\mathbf{x}, \mathbf{d}) \rightarrow (\mathbf{c}, \sigma)$$

- Sorties : volume density **σ** et couleur **RGB**

- Implémentée par un MLP

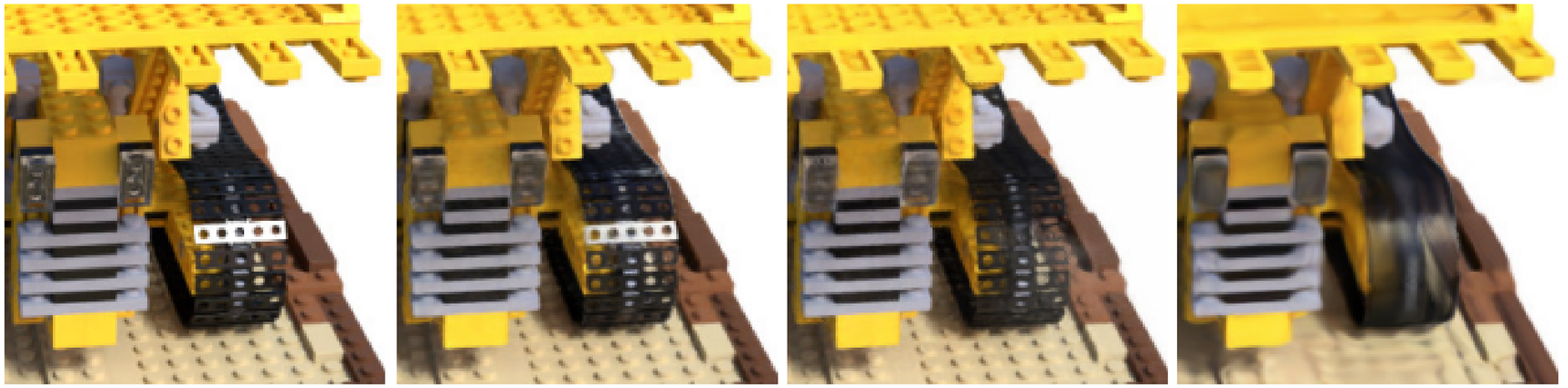
POSITIONAL ENCODING

- Objectif : Représenter les variations à haute fréquence de couleur et de géométrie

Implémentation :

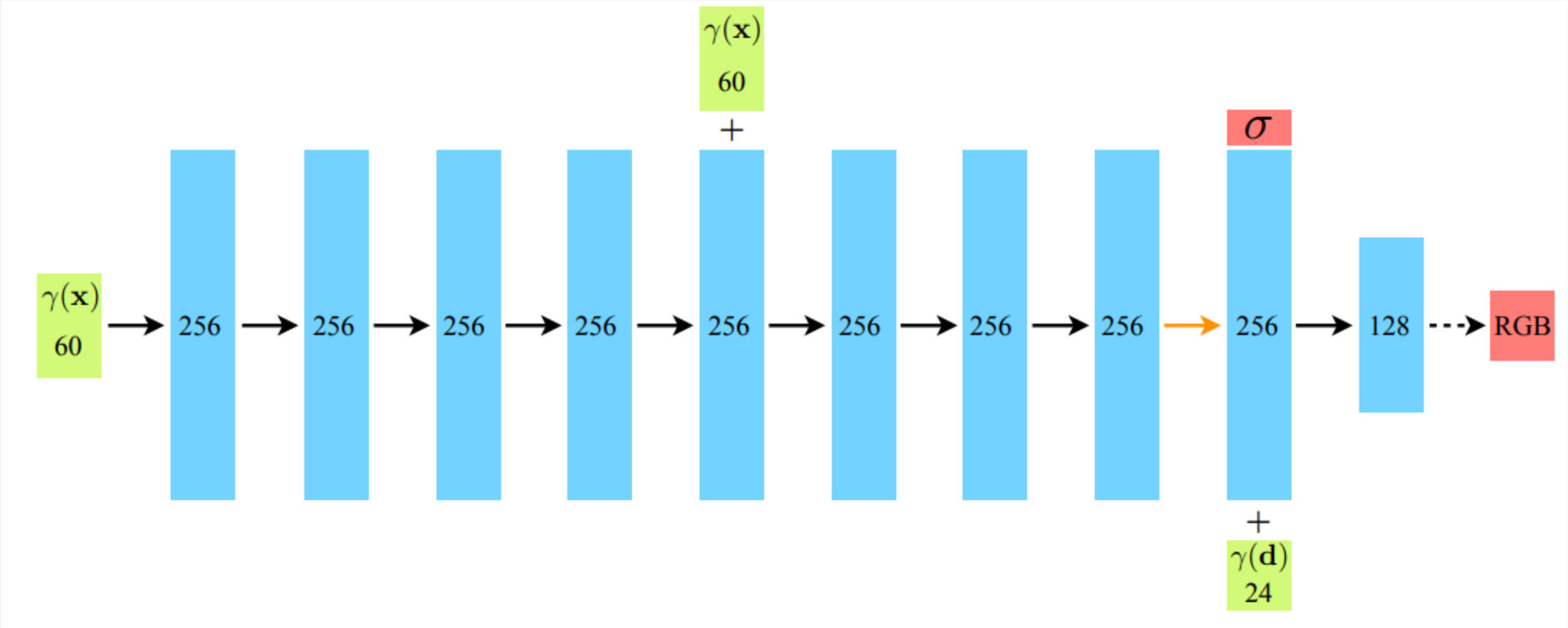
- **x** : L = 10
- **d** : L =4

$$\gamma(p) = \left(\sin\left(2^0\pi p\right), \cos\left(2^0\pi p\right), \cdots, \sin\left(2^{L-1}\pi p\right), \cos\left(2^{L-1}\pi p\right)\right)$$



Ground Truth | Complete Model | No View Dependence | No Positional Encoding

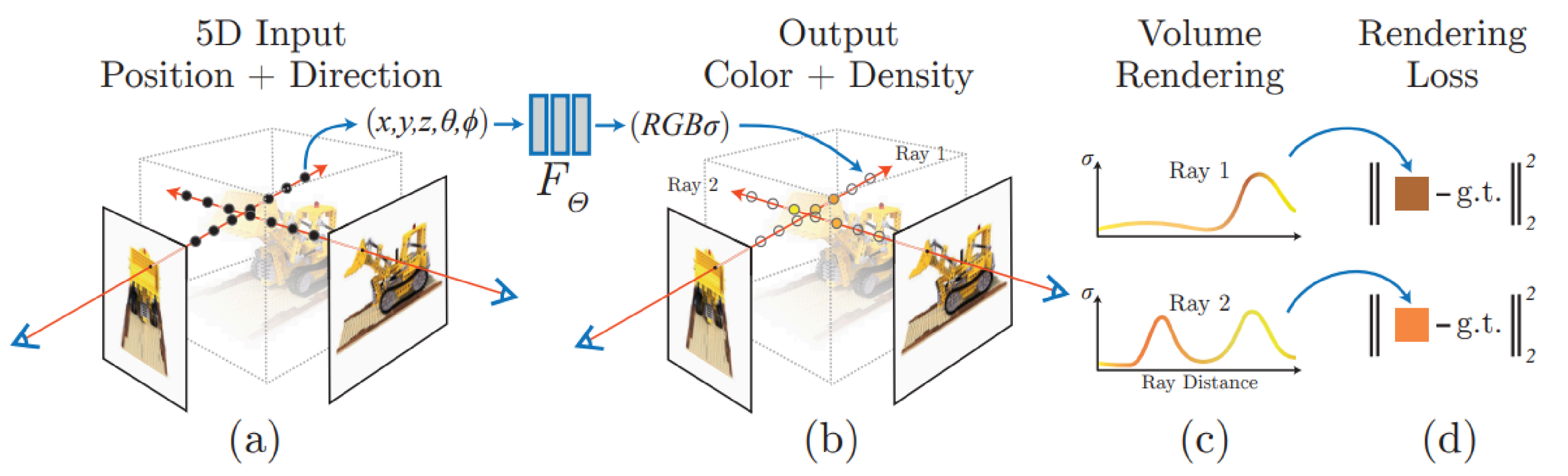
ARCHITECTURE



Flèches noires : ReLU
·+ : concaténation

Flèches orange : sans activation
Flèches pointillées : sigmoïde

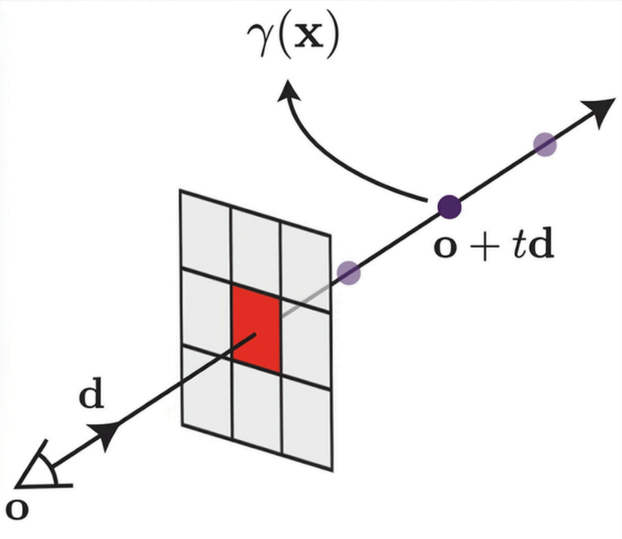
VOLUME RENDERING



VOLUME RENDERING

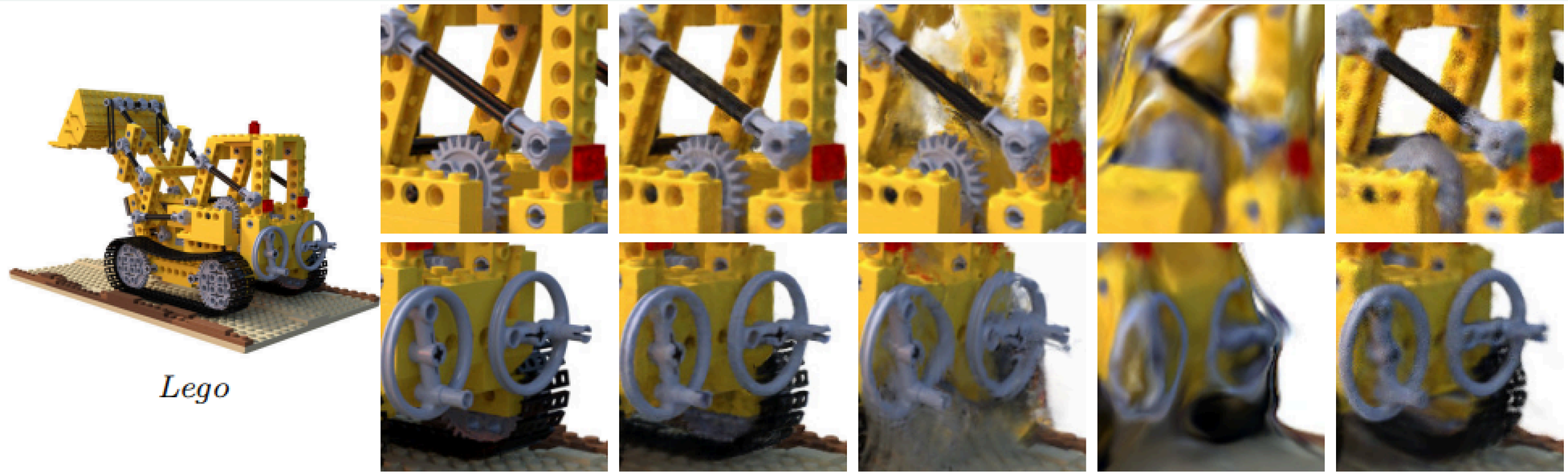
$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})dt\,, \text{ where } T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i\delta_i))\mathbf{c}_i\,, \text{ where } T_i = \exp\left(-\sum_{j=1}^{i-1} \sigma_j\delta_j\right)$$



$$\mathcal{L} = \frac{1}{N}\sum_{\mathbf{r}\in\mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2$$

RÉSULTAT

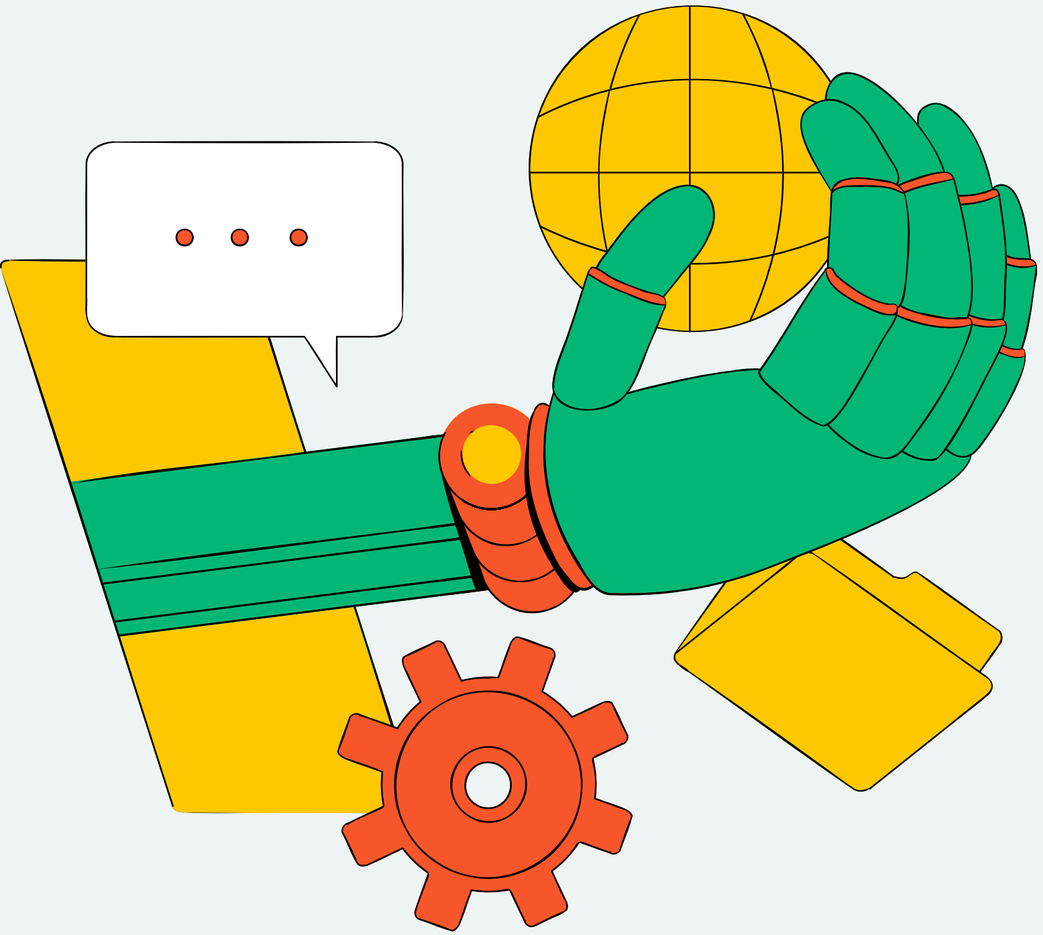


Ground Truth NeRF (ours) LLFF [28] SRN [42] NV [24]

$$\mathrm{PSNR} = 10 \cdot \log_{10}(\mathcal{L})$$

	PSNR↑							
	Chair	Drums	Ficus	Hotdog	Lego	Materials	Mic	Ship
SRN [42]	26.96	17.18	20.73	26.81	20.85	18.09	26.85	20.60
NV [24]	28.33	22.58	24.79	30.71	26.08	24.22	27.78	23.93
LLFF [28]	28.72	21.13	21.79	31.41	24.54	20.72	27.48	23.22
Ours	**33.00**	**25.01**	**30.13**	**36.18**	**32.54**	**29.62**	**32.91**	**28.65**

CODE



Tiny nerf est une implémentation minimale de nerf en TensorFlow.

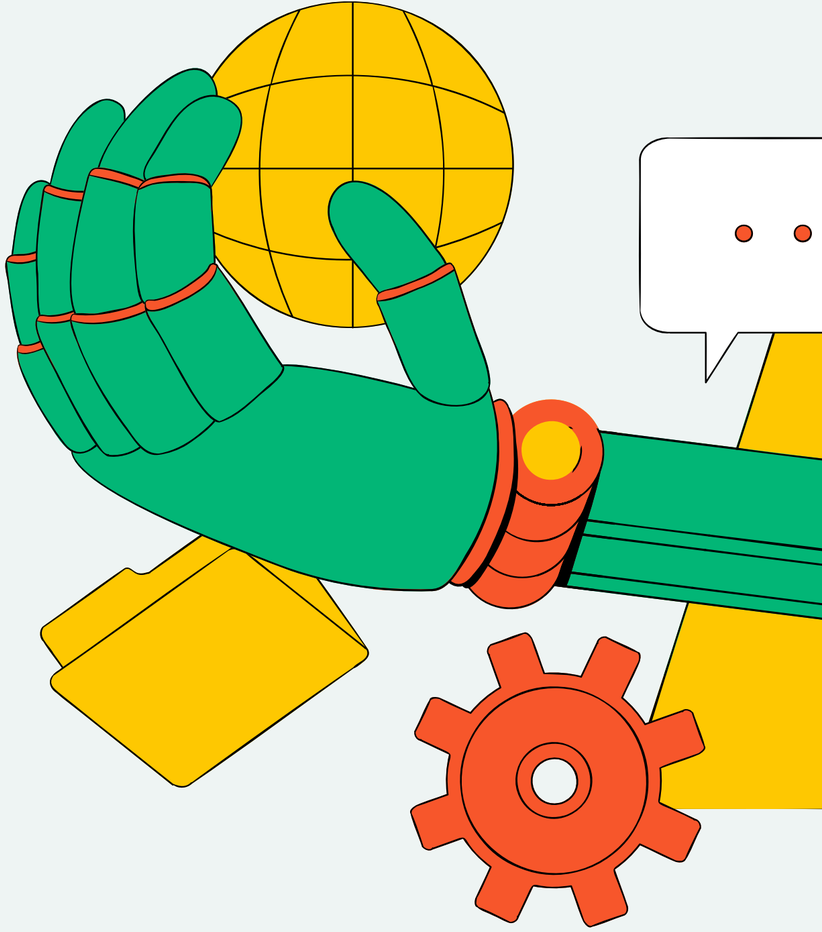
Cette implémentation n'inclut pas la direction en input (uniquement la position), ni le hierarchical sampling.

TensorFlow très proche de PyTorch

CODE

Pipeline du code :

- Chargement des données sur lesquelles on veut overfit un modèle.
- Définition du modèle, de l'encodage, get_rays et render_rays.
- Boucle d'entrainement (avec test du modèle toutes les i_plot itérations).
- Génération d'une vidéo 3D de la scène.



CODE

Classe du modèle

```python
class NeRF(nn.Module):
    def __init__(self, D=8, W=256, L_pts=10, L_views=4):
        super().__init__()
        self.L_pts = L_pts
        self.L_views = L_views

        input_dim_pts = 3 + 3 * 2 * L_pts
        input_dim_views = 3 + 3 * 2 * L_views

        # Couches pour traiter la position (x, y, z)
        self.pts_layers = nn.ModuleList([nn.Linear(input_dim_pts, W)])
        for i in range(1,D):
            if i == 4:
                self.pts_layers.append(nn.Linear(W + input_dim_pts, W))
            else:
                self.pts_layers.append(nn.Linear(W, W))

        # Sortie de la densité
        self.sigma_layer = nn.Linear(W, 1)

        # Branche pour la couleur
        self.feature_layer = nn.Linear(W, W)
        self.views_layer = nn.Linear(W + input_dim_views, W // 2)
        self.rgb_layer = nn.Linear(W // 2, 3)
```

```python
def forward(self, pts_enc, views_enc):
    # Position
    h = pts_enc
    for i, layer in enumerate(self.pts_layers):
        if i == 4:
            h = torch.cat([pts_enc, h], dim=-1)
        h = F.relu(layer(h))
    # On récupère la densité
    sigma = self.sigma_layer(h)
    # On ajoute la direction de vue pour le RGB
    feature = self.feature_layer(h)
    h = torch.cat([feature, views_enc], dim=-1)
    h = F.relu(self.views_layer(h))
    rgb = torch.sigmoid(self.rgb_layer(h))
```

CODE

Config jeu de donnée

notebooks/data/nerf_synthetic/lego/train/r_0.png

nerf-pytorch/configs/lego.txt

```
expname = blender_paper_lego
basedir = ./logs
datadir = ./data/nerf_synthetic/lego
dataset_type = blender

no_batching = True

use_viewdirs = True
white_bkgd = True
lrate_decay = 500

N_samples = 64
N_importance = 128
N_rand = 1024

precrop_iters = 500
precrop_frac = 0.5

half_res = True
```

CODE

Boucle d'entrainement

```
for i in range(N_iters + 1):

    # log_memory("iter_start")

    img_i = np.random.randint(images.shape[0])
    target = images[img_i].to(device)
    pose = poses[img_i].to(device)
    rays_o, rays_d = get_rays(H, W, focal, pose)


    coords = torch.stack(torch.meshgrid(torch.arange(H, device=device),torch.arange(W, device=device), indexing='ij'), -1).reshape(-1, 2)


    select_inds   = np.random.choice(coords.shape[0], size=[N_rand], replace=False)
    select_coords = coords[select_inds].long()
    rays_o_batch = rays_o[select_coords[:, 0], select_coords[:, 1]]
    rays_d_batch = rays_d[select_coords[:, 0], select_coords[:, 1]]
    target_s = target[select_coords[:, 0], select_coords[:, 1]]


    rgb_pred, _, _ = render_rays(model, rays_o_batch, rays_d_batch, near=2., far=6., N_samples=N_samples, rand=True)
    loss = torch.mean((rgb_pred - target_s) ** 2)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

EXPÉRIMENTATION

NeRF, 100K itérations :

Notre tiny_nerf,
100K itérations :

EXPÉRIMENTATION

	Chair	Drums	Hotdog
Attendu			
10k itérations	PSNR ≃ 21	PSNR ≃ 21	PSNR ≃ 25

CONCLUSION

- Avantages : mémoire

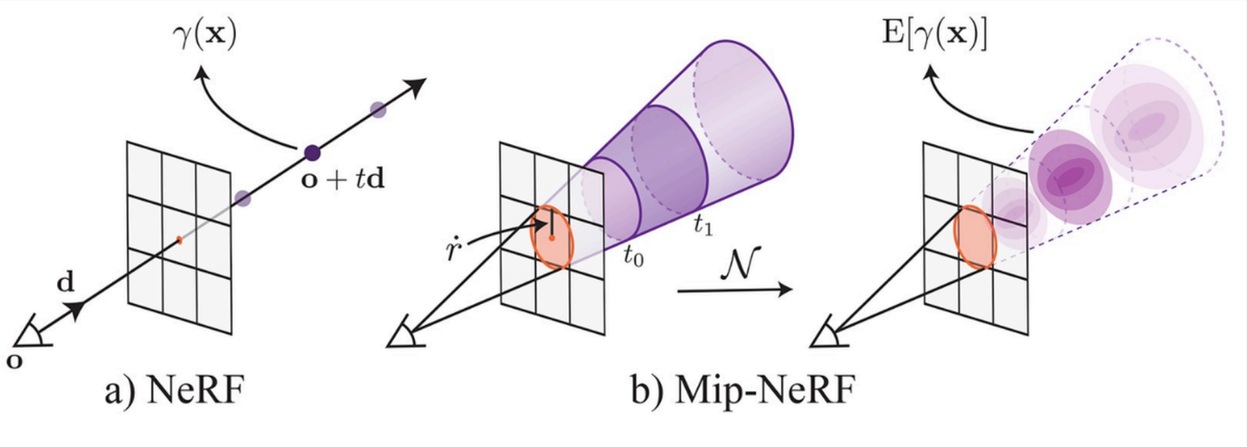
→ Représentation continue d'une scène

→ Mémoire : ~5 MB pour les poids du réseau >15 GB pour LLFF

- Limites :

→ Pas de généralisation

→ Temps : 1 à 2 jours d'entrainement



Piste d'amélioration

- Hierarchical volume sampling
- Multiscale Representation

BIBLIOGRAPHIE/SOURCES

- Repo du projet : https://github.com/dataflowr/Project-nerf-pytorch
- Page du projet : https://github.com/bmild/nerf
- Papier : https://arxiv.org/pdf/2003.08934
- Jeu de données : https://cseweb.ucsd.edu/~viscomp/projects/LF/papers/ECCV20/nerf/ https://drive.google.com/drive/folders/1cK3UDIJqKAAm7zyrxRYVFJ0BRMgrwhh4